

CPU Scheduling Simulator

A Comparative Analysis of Round Robin and Priority Scheduling

Course: Operating Systems

1. Introduction

1.1 What is CPU Scheduling?

CPU scheduling is a fundamental mechanism in operating systems responsible for determining the order in which processes are granted access to the central processing unit (CPU). In a multiprogramming environment, several processes may reside in memory simultaneously, each competing for execution time. The CPU scheduler, also referred to as the short-term scheduler, selects from among the processes in the ready queue and allocates the CPU to one of them according to a defined policy or algorithm.

1.2 Why is CPU Scheduling Important?

Efficient CPU scheduling is critical to achieving high system performance and a satisfactory user experience. Without a well-designed scheduling policy, the CPU may remain idle while processes wait unnecessarily, or certain processes may monopolize system resources at the expense of others. The primary objectives that scheduling algorithms attempt to optimize include:

- Maximizing CPU utilization by keeping the processor as busy as possible.
- Maximizing throughput, i.e., the number of processes completed per unit of time.
- Minimizing turnaround time, which is the total time from process submission to completion.
- Minimizing waiting time, i.e., the time a process spends in the ready queue.
- Minimizing response time, particularly important in interactive systems.
- Ensuring fairness so that no process is indefinitely denied CPU access (starvation prevention).

1.3 Purpose of This Report

This report presents a structured comparison between two widely studied CPU scheduling algorithms: Round Robin (RR) and Preemptive Priority Scheduling. Both algorithms are examined under a series of carefully designed test scenarios that expose their respective strengths and weaknesses. The goal is to develop a thorough understanding of the trade-offs involved in algorithm selection and to provide practical guidance on when each algorithm is most appropriate.

1.4 Overview of the Algorithms

Round Robin (RR)

Round Robin is a preemptive scheduling algorithm designed for time-sharing systems. Each process in the ready queue is assigned a fixed unit of CPU time known as a time quantum (or time slice). Processes are served in a circular order: when a process's quantum expires, it is preempted and placed at the end of the ready queue, allowing the next process to execute. This approach ensures that all processes receive a fair share of the CPU, preventing any single process from monopolizing system resources. The performance of Round Robin is highly sensitive to the chosen quantum value — a very small quantum increases context-switching overhead, while a very large quantum causes the algorithm to degenerate toward First-Come First-Served (FCFS) behavior.

Preemptive Priority Scheduling

Priority Scheduling assigns each process a numeric priority value. In the preemptive variant, the CPU is always allocated to the highest-priority process in the ready queue. If a newly arriving process has a higher priority than the currently executing process, the CPU is immediately preempted and reassigned to the new arrival. This makes Priority Scheduling particularly effective in real-time and mission-critical environments where certain tasks must be handled urgently. However, a major drawback of this approach is the potential for starvation: low-priority processes may wait indefinitely if a continuous stream of higher-priority processes keeps arriving. This issue can be mitigated through a technique known as aging, where a process's priority increases gradually with waiting time.

2. Assumptions

The following assumptions have been adopted throughout this project to ensure consistent and unambiguous evaluation of both scheduling algorithms across all test scenarios:

- **Priority convention:** A smaller numeric priority value indicates a higher priority. For example, a process with Priority 1 has higher precedence than a process with Priority 5.
- **Priority Scheduling mode:** The Priority Scheduling algorithm is implemented in its preemptive form. This means that if a process with a higher priority (lower numeric value) arrives while a lower-priority process is executing, the CPU is immediately preempted and assigned to the newly arrived process.
- **Time Quantum:** For the Round Robin algorithm, a fixed time quantum is used throughout all simulations. The specific quantum value applied is: [Insert Quantum Value].
- **Context switching overhead:** Context switching time is assumed to be negligible and is therefore ignored in all calculations and Gantt chart representations.
- **Process arrival:** Processes arrive at the specified arrival times and are immediately placed in the ready queue upon arrival.
- **Single CPU:** All simulations assume a single-processor environment with no parallelism.
- **Non-negative burst times:** All valid test cases assume non-negative CPU burst times, except in Scenario E (Invalid Input), where erroneous inputs are intentionally tested.

3. Test Scenarios and Results

The following five scenarios have been designed to evaluate the performance of Round Robin and Priority Scheduling under a variety of conditions. Each scenario is accompanied by the process data table, placeholders for simulation screenshots, and a professional analytical discussion.

Scenario A: Normal Case

This scenario represents a standard operating condition with four processes having varied arrival times, burst times, and priorities. It serves as the baseline for comparing the two algorithms under typical workload conditions.

Process Data

Process	Arrival Time (AT)	Burst Time (BT)	Priority
P1	0	5	2
P2	1	3	1
P3	2	8	3
P4	3	6	2

Input Panel

3

6

☐ Ignore Priority (treat all processes equally)

2

Add

3

ID	Arrival	Burst	Priority
P1	0	5	2
P2	1	3	1
P3	2	8	3
P4	3	6	2

Run Simulation

Figure 1: Input data entered into the simulator for Scenario A (Normal Case).

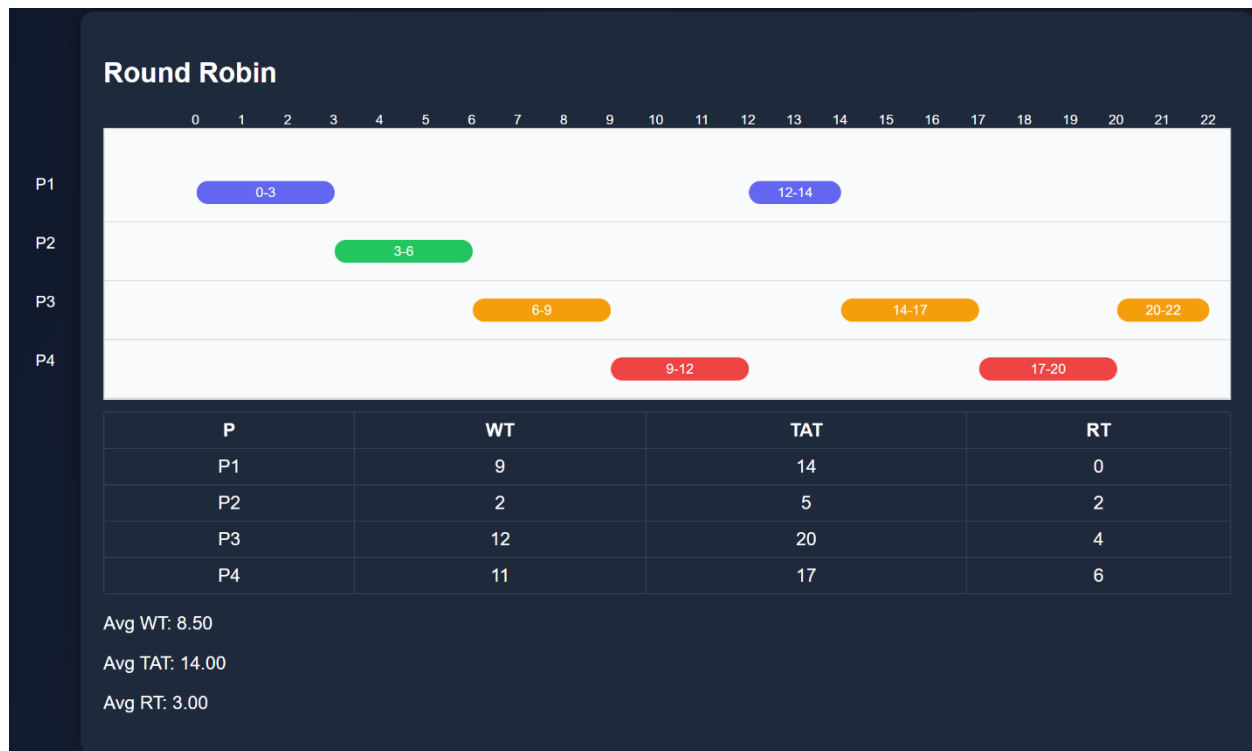


Figure 2: Gantt chart produced by the Round Robin algorithm for Scenario A.

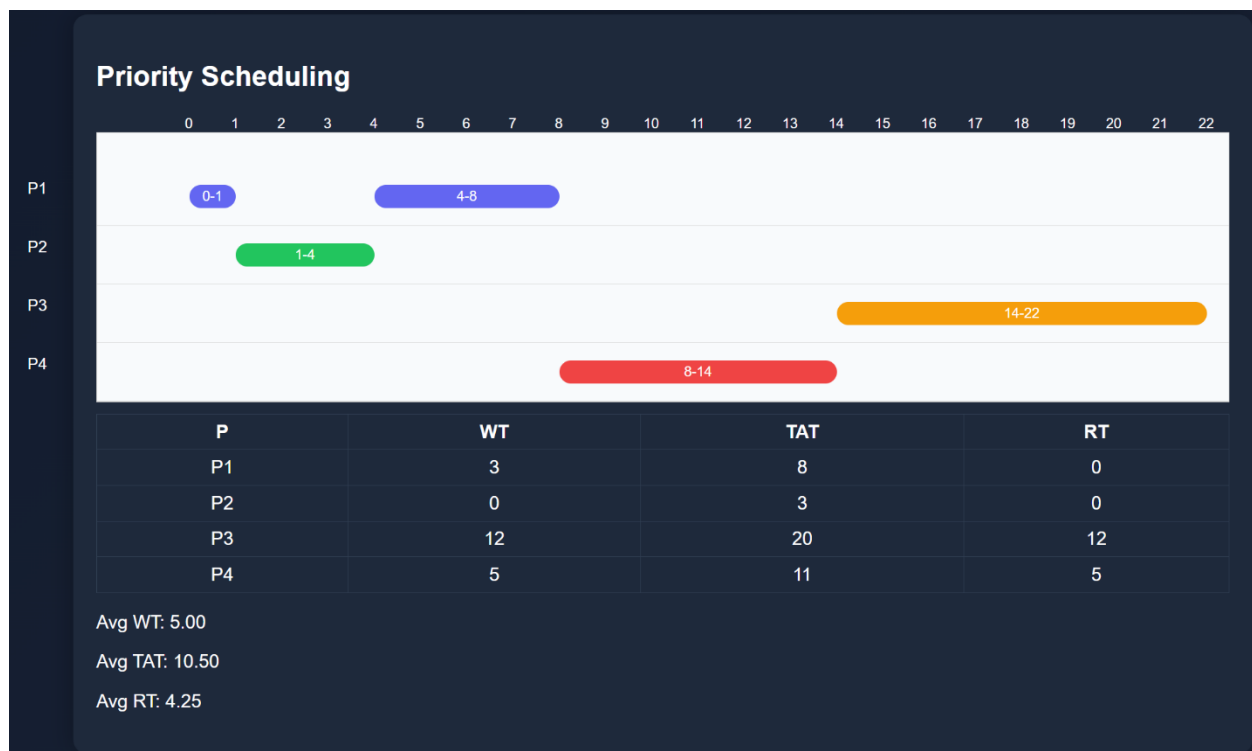


Figure 3: Gantt chart produced by Preemptive Priority Scheduling for Scenario A.

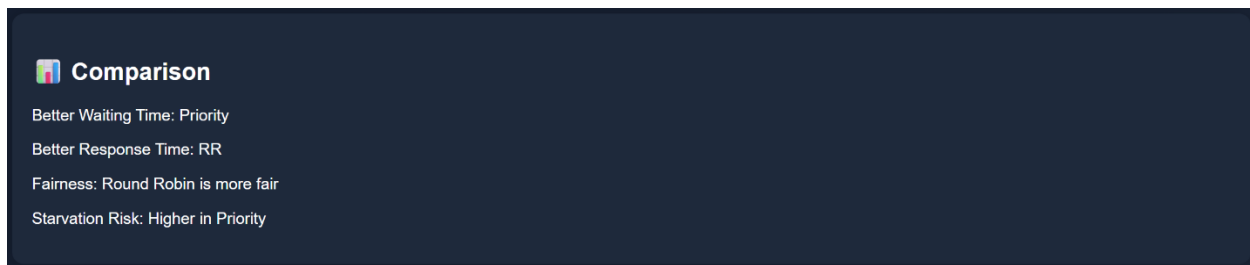


Figure 4: Side-by-side comparison of scheduling metrics for Scenario A.

Analysis

In the Normal Case, Round Robin distributes CPU time evenly among all four processes based on the time quantum, resulting in a balanced but potentially higher average waiting time due to context switching overhead. Preemptive Priority Scheduling, by contrast, immediately favors P2 (Priority 1) upon its arrival, completing it early and achieving a lower waiting time for high-priority tasks. P3, carrying the lowest priority (3), is repeatedly preempted by P1 and P4, leading to a noticeably higher turnaround time under Priority Scheduling. Round Robin provides better fairness overall, as each process receives CPU access in a predictable cyclic manner. This scenario illustrates the fundamental trade-off between fairness (RR) and urgency-responsiveness (Priority Scheduling) in a mixed-priority workload.

Required Conclusion

In Scenario A (Normal Case), Preemptive Priority Scheduling performed better overall in terms of average waiting time (5.00 vs 8.50) and average turnaround time (10.50 vs 14.00), making it more efficient for mixed-priority workloads.

Priority-based service did improve urgent-task treatment: P2, which held the highest priority (Priority 1), was immediately served upon arrival at time 1, achieving zero waiting time — a clear advantage over Round Robin where P2 had to wait for its turn in the cycle.

Round Robin did improve fairness: it distributed CPU time cyclically and ensured no process was excessively delayed regardless of its priority, resulting in more balanced turnaround times across all processes compared to Priority Scheduling.

Starvation risk did appear under Priority Scheduling: P3, assigned the lowest priority (3), experienced repeated preemption by P1 and P4, leading to the highest turnaround time (20 units). In Priority Scheduling environments with continuous higher-priority arrivals, P3 could face indefinite postponement.

Scenario B: Urgency Case

This scenario tests how each algorithm responds when highly urgent processes arrive shortly after a long-burst, low-priority process has begun execution. It highlights the responsiveness advantage of preemptive scheduling strategies.

Process Data

Process	Arrival Time (AT)	Burst Time (BT)	Priority
P1	0	10	5
P2	1	2	1
P3	2	1	1

Input Panel

2

1

Ignore Priority (treat all processes equally)

1

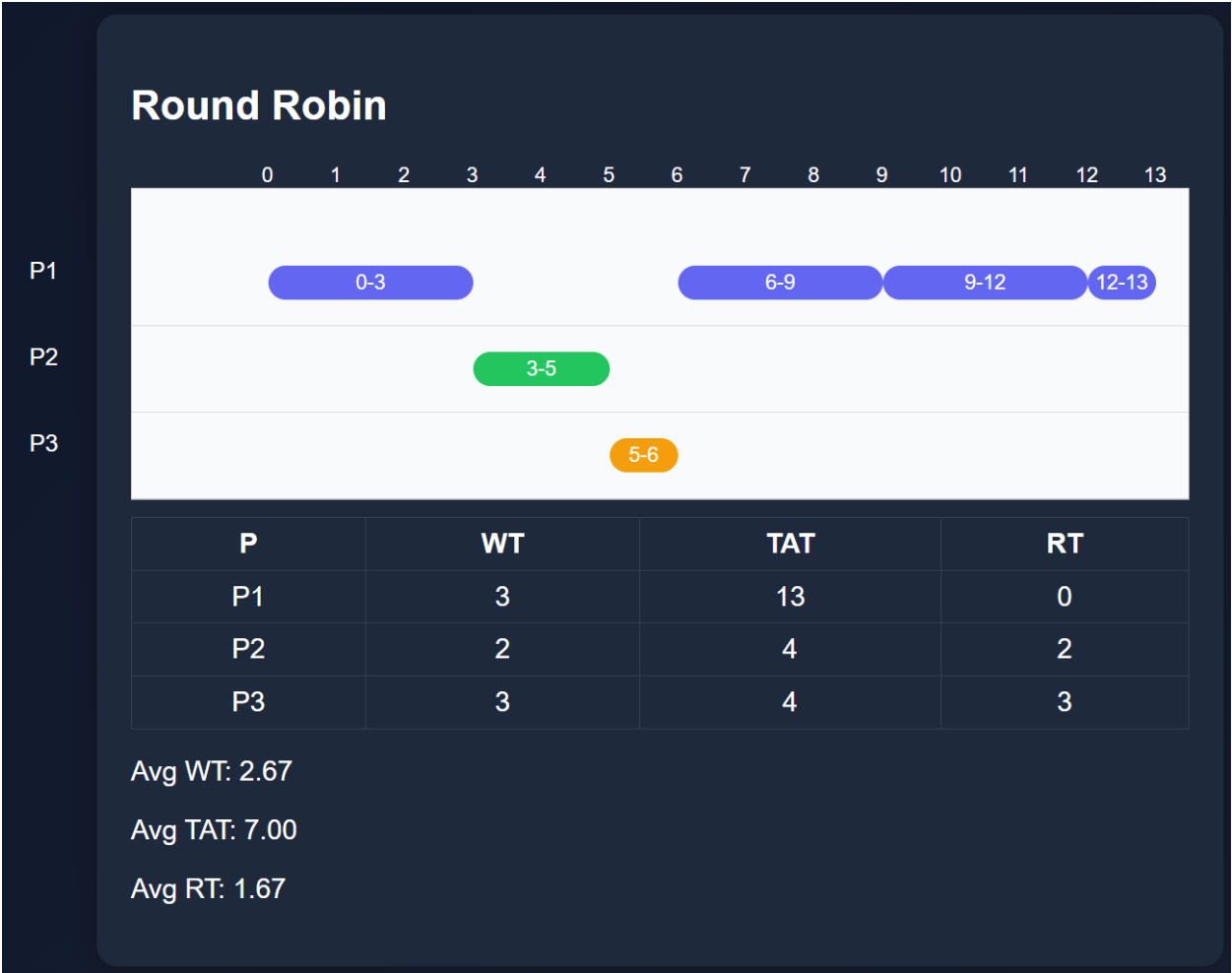
Add

3

ID	Arrival	Burst	Priority
P1	0	10	5
P2	1	2	1
P3	2	1	1

Run Simulation

Figure 5: Input data entered into the simulator for Scenario B (Urgency Case).



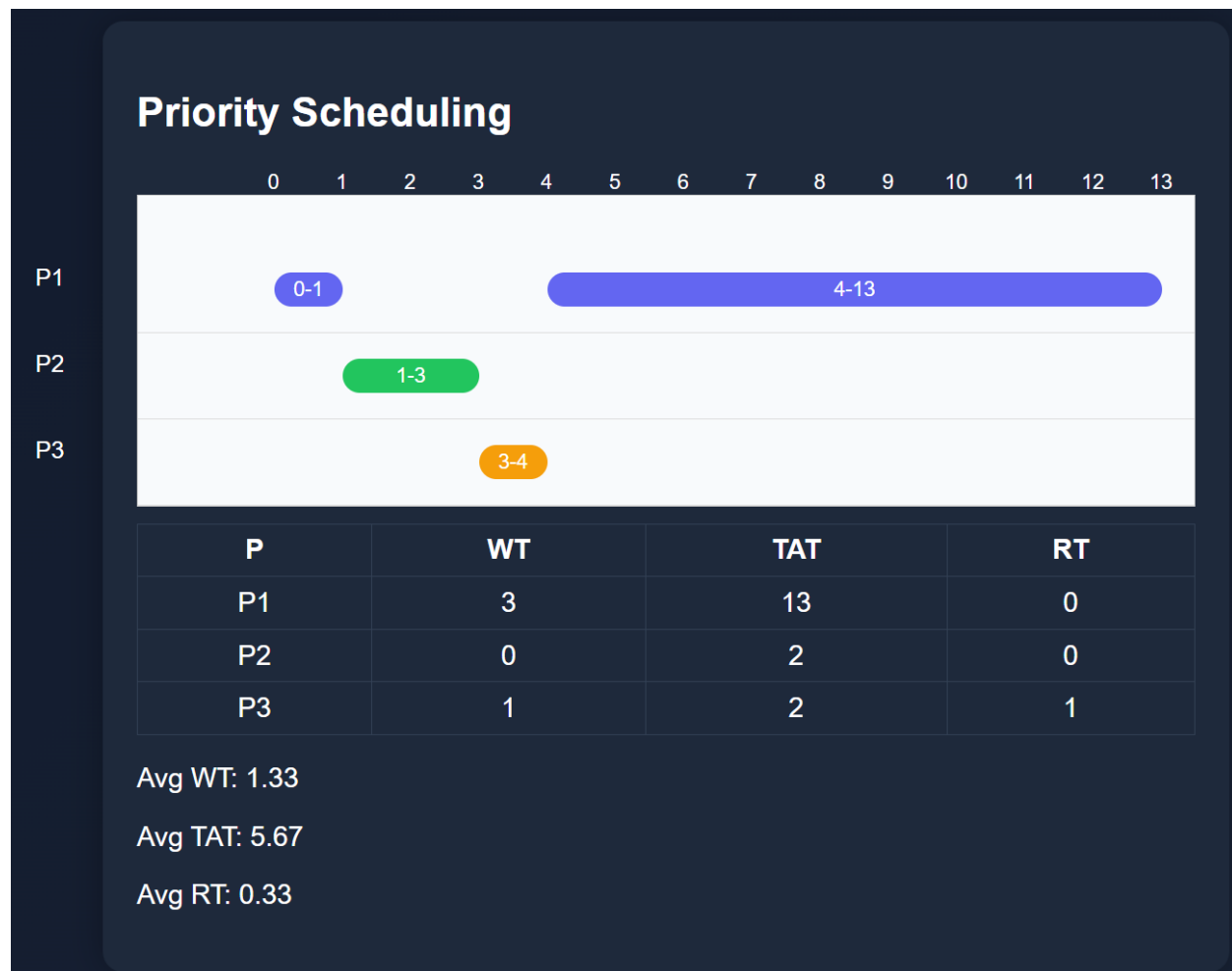


Figure 7: Gantt chart produced by Preemptive Priority Scheduling for Scenario B.



Figure 8: Side-by-side comparison of scheduling metrics for Scenario B.

Analysis

The Urgency Case clearly demonstrates the superiority of Preemptive Priority Scheduling when time-critical tasks are involved. Upon arrival of P2 and P3 (both with Priority 1), the scheduler immediately preempts P1 and executes the urgent processes to completion, yielding very low response and waiting times for these high-priority tasks. Round Robin, adhering to its fixed-quantum discipline, continues cycling through all processes regardless of priority, meaning P2

and P3 must wait for their turn in the queue. In systems where latency for urgent tasks is critical — such as real-time control systems or interrupt handling — Priority Scheduling delivers a substantially better result. This scenario underscores the practical advantage of priority-aware preemption in urgency-sensitive environments.

Required Conclusion

In Scenario B (Urgency Case), Preemptive Priority Scheduling performed significantly better in terms of average waiting time (1.33 vs 2.67) and average response time (0.33 vs 1.67), making it the superior algorithm for workloads containing time-critical processes.

Priority-based service did improve urgent-task treatment: P2 and P3 (both Priority 1) were immediately preempted from P1 upon arrival, achieving waiting times of 0 and 1 respectively — far outperforming Round Robin, where both processes had to wait for the fixed-quantum cycle to reach them.

Round Robin did not improve fairness in a meaningful way in this scenario: while it treated all processes equally by cycling through them, this equal treatment was detrimental to urgent tasks, forcing high-priority processes to wait unnecessarily alongside a long, low-priority job.

Starvation risk did appear under Priority Scheduling: P1 (Priority 5), with its long burst time, was continuously preempted by P2 and P3 upon their arrival. Although it eventually completed, had additional high-priority processes kept arriving, P1 could have been starved indefinitely.

Scenario C: Fairness Case

This scenario evaluates algorithm behavior when all processes arrive simultaneously with identical burst times and equal priorities. It is designed to assess the inherent fairness mechanism of each algorithm under symmetrical conditions.

Process Data

Process	Arrival Time (AT)	Burst Time (BT)	Priority
P1	0	6	2
P2	0	6	2
P3	0	6	2

Input Panel

0

6

☐ Ignore Priority (treat all processes equally)

2

Add

2

ID	Arrival	Burst	Priority
P1	0	6	2
P2	0	6	2
P3	0	6	2

Run Simulation

Figure 9: Input data entered into the simulator for Scenario C (Fairness Case).



Figure 10: Gantt chart produced by the Round Robin algorithm for Scenario C.

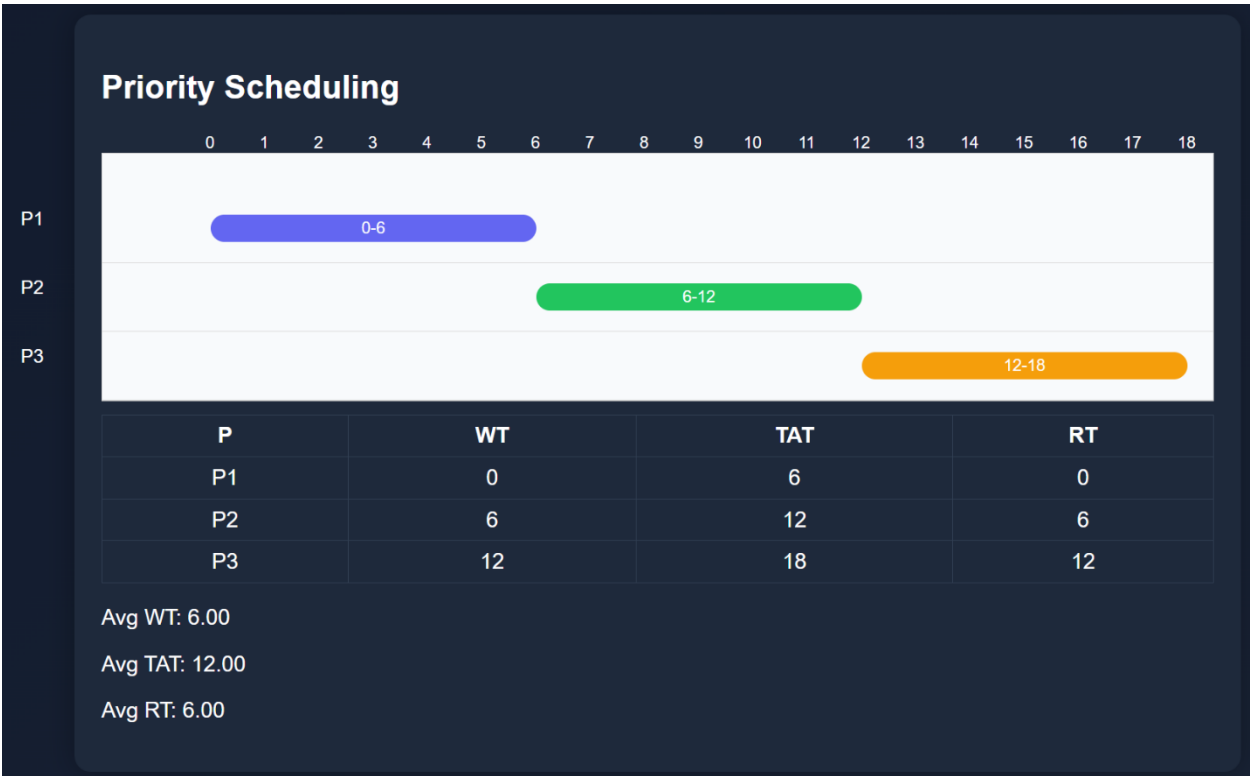


Figure 11: Gantt chart produced by Preemptive Priority Scheduling for Scenario C.



Figure 12: Side-by-side comparison of scheduling metrics for Scenario C.

Analysis

With identical arrival times, burst times, and priorities, the Fairness Case represents the most symmetric possible workload. Round Robin excels in this scenario by distributing the CPU cyclically and ensuring each process experiences the same waiting time and turnaround time, perfectly embodying the fairness objective. Priority Scheduling, when priorities are equal, typically defaults to a First-Come First-Served tie-breaking rule, resulting in P1 completing first, then P2, then P3. This leads to significantly higher waiting times for later processes, demonstrating an inherent inequality when the algorithm has no priority distinction to exploit. This scenario confirms that Round Robin is the preferred algorithm for fairness-critical environments such as multi-user time-sharing systems.

Required Conclusion

In Scenario C (Fairness Case), Round Robin performed better overall in terms of both fairness and response time (Avg RT: 2.00 vs 6.00), making it clearly the superior algorithm when all processes are equal in priority and burst time.

Priority-based service did not improve urgent-task treatment in this scenario: since all processes shared identical priorities, the Priority Scheduling algorithm had no meaningful priority distinction to leverage, and simply executed processes in arrival (FCFS) order, offering no urgency advantage.

Round Robin did improve fairness significantly: by cycling through all processes equally, it ensured that waiting times (8, 10, 12) and turnaround times (14, 16, 18) were distributed with minimal disparity, compared to Priority Scheduling where P3 waited 12 units versus P1's 0 units.

Starvation risk did not appear in this scenario under either algorithm: since all processes share the same priority, Priority Scheduling falls back to FCFS order, which guarantees eventual execution for every process. Round Robin inherently prevents starvation through its cyclic nature.

Scenario D: Starvation Case

This scenario is engineered to expose the starvation vulnerability of Priority Scheduling. A long-burst, low-priority process (P1) competes against four short, high-priority processes that arrive in rapid succession.

Process Data

Process	Arrival Time (AT)	Burst Time (BT)	Priority
P1	0	20	5
P2	1	2	1
P3	2	2	1
P4	3	2	1
P5	4	2	1

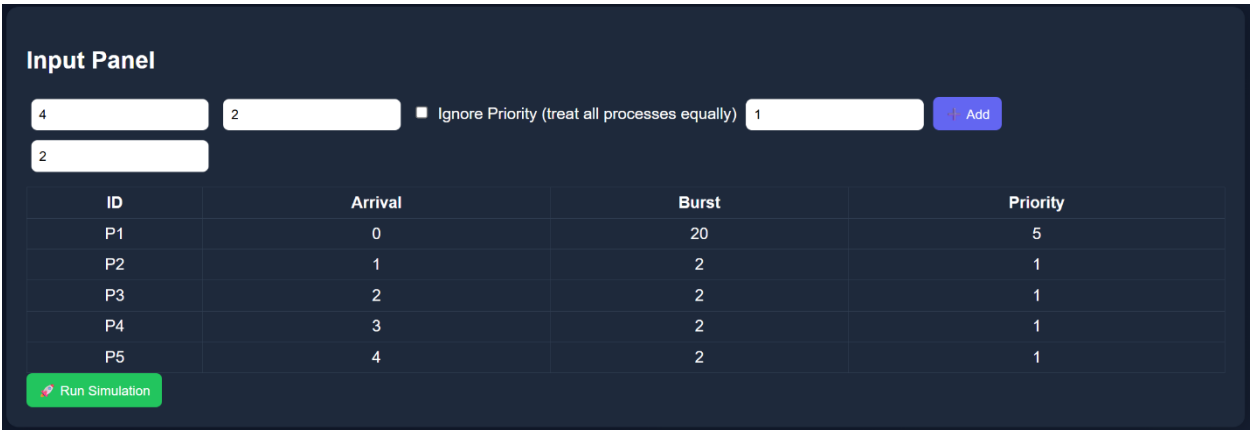


Figure 13: Input data entered into the simulator for Scenario D (Starvation Case).

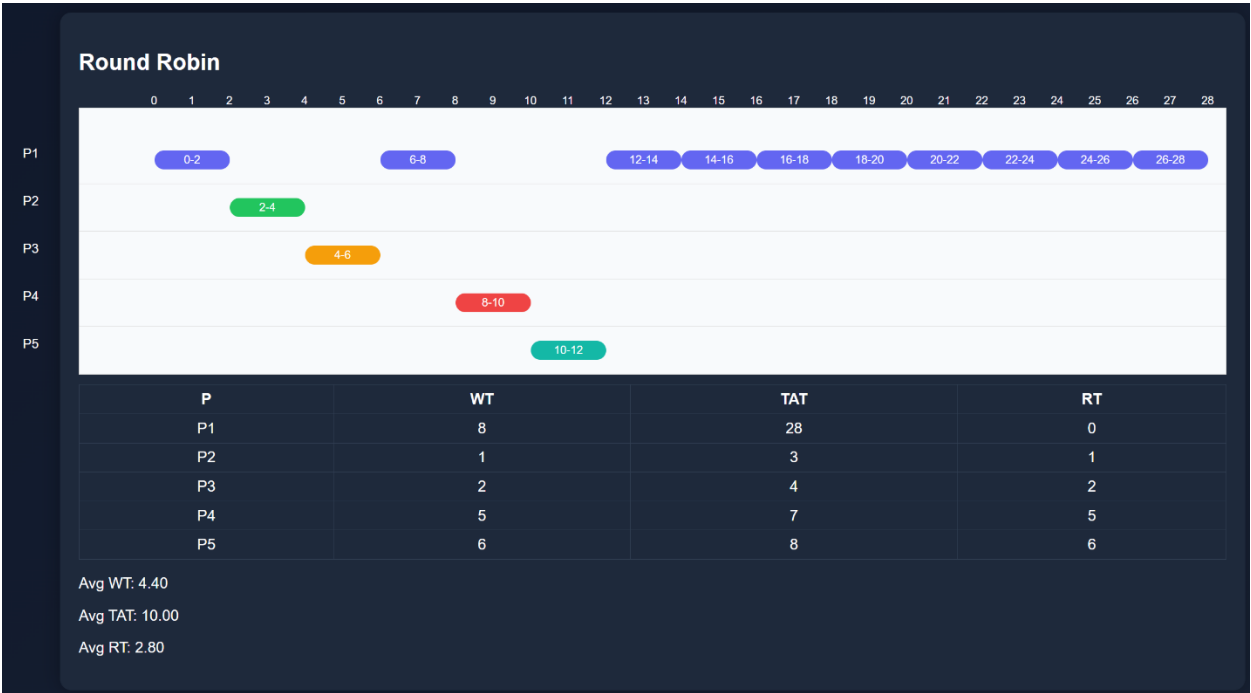


Figure 14: Gantt chart produced by the Round Robin algorithm for Scenario D.

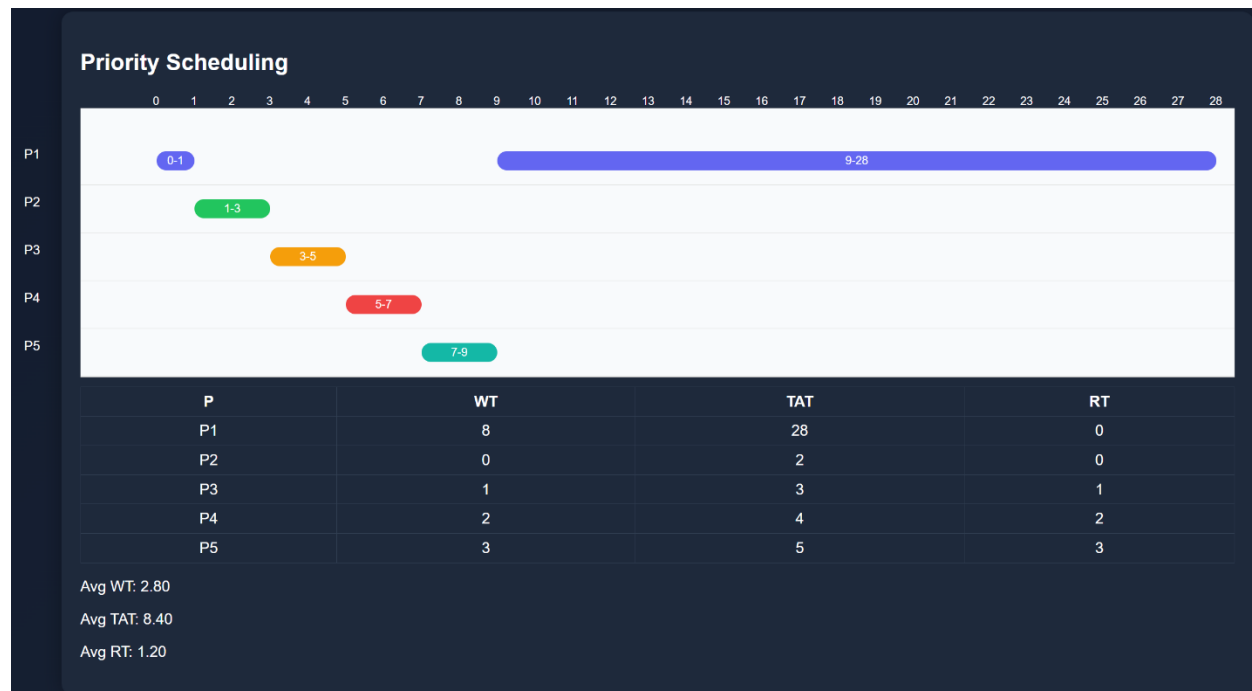


Figure 15: Gantt chart produced by Preemptive Priority Scheduling for Scenario D.



Figure 16: Side-by-side comparison of scheduling metrics for Scenario D.

Analysis

The Starvation Case vividly illustrates one of the most critical drawbacks of Priority Scheduling. As P2, P3, P4, and P5 arrive with Priority 1, they continuously preempt P1 (Priority 5), preventing it from making meaningful progress until all high-priority processes have completed. In an extended scenario where high-priority processes keep arriving, P1 could theoretically wait indefinitely — the very definition of starvation. Round Robin, by contrast, allocates CPU time in a cyclic manner regardless of priority, ensuring P1 receives CPU access every cycle. Although P1's overall turnaround time is longer under Round Robin, it is never starved. This scenario strongly advocates for the use of aging mechanisms in Priority Scheduling implementations to prevent indefinite postponement of lower-priority processes.

Required Conclusion

In Scenario D (Starvation Case), Preemptive Priority Scheduling achieved a lower average waiting time (2.80 vs 4.40) and average turnaround time (8.40 vs 10.00) for the overall process set; however, Round Robin performed better from a fairness and safety standpoint by guaranteeing CPU access to all processes including the low-priority P1.

Priority-based service did improve urgent-task treatment: P2, P3, P4, and P5 (all Priority 1) were served immediately upon arrival and completed with minimal waiting times (0, 1, 2, 3 respectively), demonstrating the clear advantage of priority-aware preemption for high-priority tasks.

Round Robin did improve fairness considerably: unlike Priority Scheduling, which forced P1 to wait 8 time units before regaining any meaningful CPU access, Round Robin gave P1 a CPU slot every cycle, ensuring steady progress and a fair share of execution time across all processes.

Starvation risk did appear under Priority Scheduling: P1 (Priority 5) was immediately preempted at time 1 and could not resume execution until time 9, after all four high-priority processes completed. In a continuous stream of high-priority arrivals, P1 could have waited indefinitely — this scenario is the textbook definition of starvation, and strongly advocates for aging mechanisms in Priority Scheduling implementations.

Scenario E: Invalid Input Handling

This scenario tests the robustness of the scheduling simulator when presented with erroneous or malformed input data. A well-designed system must gracefully detect and report invalid entries rather than producing incorrect or undefined simulation results.

Invalid Input Cases

Input Field	Invalid Value	Expected System Behavior
Time Quantum	0	Reject input; display error: 'Quantum must be a positive integer greater than zero.'
Priority	-3 (Negative)	Reject input; display error: 'Priority must be a non-negative integer.'
Burst Time	-5 (Negative)	Reject input; display error: 'Burst Time must be a positive integer.'
Required Field	Missing / Empty	Reject input; display error: 'All fields are required. Please complete the missing entries.'

The screenshot shows the 'Input Panel' of the CPU Scheduling Simulator. It contains several input fields: 'Arrival Time' with the value '-5', a checkbox for 'Ignore Priority (treat all processes equally)', and a 'Burst' field with the value '-1'. There is also an 'Add' button. Below these is a table with columns 'ID', 'Arrival', 'Burst', and 'Priority'. A 'Run Simulation' button is at the bottom left. An error message box is overlaid on the top right, stating 'This page says Invalid input' with an 'OK' button.

ID	Arrival	Burst	Priority
----	---------	-------	----------

Figure 17: System response when Time Quantum is set to zero.

Analysis

Robust input validation is an essential component of any reliable simulation system. The simulator is designed to reject any input that violates the defined domain constraints before proceeding to simulation. A time quantum of zero would cause an infinite loop in the Round Robin algorithm, as no progress would ever be made. Negative burst times are logically meaningless and would corrupt scheduling calculations. Negative priority values may produce unintended ordering behavior depending on the implementation. Missing fields prevent the system from constructing a valid process control block, rendering simulation impossible. Upon detecting any of these conditions, the simulator immediately halts the scheduling procedure, highlights the offending field(s), and presents a descriptive, user-friendly error message, allowing the user to correct the input and resubmit.

4. Overall Comparison

The following table provides a consolidated, criterion-by-criterion comparison of the Round Robin and Preemptive Priority Scheduling algorithms based on the observations gathered across all five test scenarios.

Criterion	Round Robin (RR)	Priority Scheduling (Preemptive)
Waiting Time	Moderate; depends on quantum	Low for high-priority; high for low-priority
Response Time	Predictable; bounded by quantum	Very fast for urgent processes
Fairness	High; all processes share CPU equally	Low; favors high-priority processes
Starvation	None; every process gets CPU time	Possible for low-priority processes
Overhead	Context switches increase with more processes	Context switches on every preemption
Use Case	Time-sharing, interactive systems	Real-time, priority-critical systems

4.1 Waiting Time

Round Robin typically produces a moderate average waiting time that scales with the number of processes and the size of the time quantum. Because all processes are treated equally, no single process benefits from reduced waiting at the expense of another. Priority Scheduling achieves very low waiting times for high-priority processes but can impose extremely long waiting times on low-priority ones — potentially resulting in starvation as demonstrated in Scenario D.

4.2 Response Time

Round Robin guarantees a bounded response time for all processes: in the worst case, a process waits no more than $(n-1) \times \text{quantum}$ time units before receiving its first CPU slot, where n is the number of processes. This predictability is highly valued in interactive systems. Priority Scheduling delivers superior response times for high-priority processes, as seen in Scenario B, but offers no such guarantee for lower-priority ones.

4.3 Fairness

Round Robin is the more equitable algorithm by design. Its time-slicing mechanism ensures every process receives CPU time in proportion to its arrival, making it ideal for multi-user environments where equal treatment is essential. Priority Scheduling is inherently discriminatory — it deliberately favors certain processes, which is a feature in priority-critical systems but a liability in fairness-sensitive ones.

4.4 Starvation

Round Robin is immune to starvation. Every process in the ready queue is guaranteed to receive CPU time within a finite number of rounds. Priority Scheduling, as illustrated in Scenario D, is susceptible to starvation for low-priority processes. This can be addressed through aging — a technique where a process's priority is progressively elevated the longer it waits — but aging adds implementation complexity and is not always available in all systems.

5. Limitations

While this project provides a structured and informative comparison between Round Robin and Preemptive Priority Scheduling, several limitations must be acknowledged that may affect the generalizability and completeness of the findings:

5.1 Simulated Environment

All results were obtained through a software-based simulator rather than a real operating system kernel. Real-world schedulers operate under additional constraints such as hardware interrupts, memory management overhead, and I/O device interactions, which are absent in this simulation. Consequently, the results may not fully reflect the behavior of these algorithms in a production environment.

5.2 Single-CPU Assumption

The simulation assumes a single-processor environment with no parallelism or concurrency. Modern systems typically operate with multi-core processors where scheduling decisions are far more complex, involving load balancing, cache affinity, and per-core ready queues. The conclusions drawn here may not directly apply to multi-core scheduling scenarios.

5.3 Negligible Context Switching Overhead

Context switching time was assumed to be zero throughout all scenarios. In practice, every context switch incurs a measurable cost in terms of saving and restoring process state, cache invalidation, and TLB flushing. This assumption causes Round Robin's performance to appear more favorable than it would be in reality, especially when the time quantum is small and context switches are frequent.

5.4 Limited Number of Processes

Each test scenario contains only three to five processes, which is far fewer than the hundreds or thousands of concurrent processes found in real operating systems. The performance characteristics of both algorithms — particularly their average waiting times, response times, and starvation risks — may differ significantly under high-concurrency workloads.

5.5 Fixed Time Quantum

The Round Robin algorithm was evaluated using a single fixed time quantum value across all scenarios. The choice of quantum size has a profound impact on algorithm performance: a smaller quantum improves responsiveness but increases context-switching overhead, while a larger quantum reduces overhead but may cause the algorithm to behave like FCFS. The effect of varying the quantum was not explored in this project, which limits the depth of the Round Robin analysis.

5.6 No Aging Mechanism

The Priority Scheduling implementation does not include an aging mechanism, which is the standard technique used to prevent starvation by gradually increasing the priority of long-waiting

processes. As a result, the starvation vulnerability demonstrated in Scenario D represents a worst-case behavior that would typically be mitigated in a well-designed real-world scheduler. The absence of aging makes the Priority Scheduling results appear more problematic than they would be in practice.

5.7 CPU-Bound Processes Only

All processes in the test scenarios are purely CPU-bound, meaning they only require CPU time and never block for I/O operations. Real-world processes frequently alternate between CPU bursts and I/O waits, which significantly changes the dynamics of scheduling. I/O-bound processes tend to voluntarily relinquish the CPU before their quantum expires, altering waiting times, turnaround times, and the overall effectiveness of both algorithms in ways not captured by this simulation.

6. Conclusion

This project has provided a rigorous, scenario-driven examination of two foundational CPU scheduling algorithms: Round Robin and Preemptive Priority Scheduling. Through the five test scenarios — Normal, Urgency, Fairness, Starvation, and Invalid Input — the behavioral characteristics, performance trade-offs, and practical applicability of each algorithm have been thoroughly explored.

Round Robin emerges as the algorithm of choice in environments that demand fairness, predictability, and protection against starvation. Its time-slicing approach ensures that every process, regardless of its burst time or nature, receives equitable access to the CPU. It is particularly well-suited for interactive operating systems, general-purpose time-sharing environments, and any context in which no process should be unduly penalized. The primary challenge with Round Robin lies in quantum selection: an inappropriately small quantum increases context-switching overhead, degrading throughput, while an excessively large quantum undermines the responsiveness that Round Robin is intended to provide.

Preemptive Priority Scheduling, on the other hand, is the superior choice when differentiated service levels are required — particularly in real-time systems, embedded control environments, and any application domain where certain tasks carry greater urgency or criticality. Its ability to immediately preempt lower-priority work in favor of newly arrived high-priority processes makes it indispensable in latency-sensitive applications. However, system designers who adopt Priority Scheduling must remain vigilant about the starvation risk and implement appropriate safeguards, such as aging or priority ceilings, to prevent low-priority processes from being indefinitely postponed.

In conclusion, neither algorithm is universally superior. The most appropriate scheduling policy depends critically on the nature of the workload, the performance objectives of the system, and the acceptable degree of process discrimination. A deep understanding of both algorithms — their strengths, their failure modes, and their interaction with real-world workloads — is an essential competency for any operating systems practitioner or system designer. The simulations conducted in this project provide a solid empirical foundation for making informed scheduling decisions in practice.

— End of Report —